

COMPUTER PROGRAMMING PARADIGMS AND FUNDAMENTALS

Chapters 3 – 6

Chapter 3: Conditional Statements

Chapter 4: Controlling Program Flow with Loops

Chapter 5: Introduction to Objects and Classes

Chapter 6: String Processing and Simple I/O

CHAPTER THREE

CONDITIONAL STATEMENTS

3.1 Introduction to Conditional Statements

Conditional statements allow a program to make decisions based on certain conditions. In Java, the primary conditional constructs are the if statement, the if-else statement, nested if statements, and the switch statement. These constructs evaluate boolean expressions and execute different blocks of code depending on whether the expression is true or false.

3.2 The if Statement

The if statement executes a block of code only when a specified condition is true. If the condition evaluates to false, the block is skipped entirely.

Syntax:

```
if (condition) {  
    // statements to execute if condition is true  
}
```

Example:

If a student's score is 70 or above, print a passing message:

```
int score = 75;  
if (score >= 70) {  
    System.out.println("Passing grade");  
}
```

3.3 The if-else Statement

The if-else statement provides an alternative block of code that executes when the condition is false. This allows the program to choose between two possible paths.

Syntax:

```
if (condition) {  
    // executes if condition is true  
} else {  
    // executes if condition is false  
}
```

Example: Checking if a number is even or odd:

```
int number = 7;  
if (number % 2 == 0) {
```

```

    System.out.println("Even");
} else {
    System.out.println("Odd");
}

```

Comparison: if vs if-else

Feature	if	if-else
Number of Conditions	One condition	One condition (two outcomes)
Alternative Path	None	Has one alternative path (else)
Use Case Example	Check if positive; only act if true	Check if a number is even or odd
Execution Flow	If false, nothing happens	If false, the else block executes
Code Complexity	Simple	Slightly more complex
Best For	Single checks	Binary decisions

Best Practices for Avoidance:

- Maintain consistent indentation.
- Use explicit block delimiters where applicable.
- Avoid excessive nesting.
- Add comments to clarify logic.

Corrected Version:

```

if (condition1) {
    print("Action A")
} else {
    print("Action B")
}

```

In this example, the else is automatically associated with condition1, not condition2.

3.3 Nested if Statements and the Dangling Else Problem

3.3.1 Introduction to Nested if Statements

Nested if statements are an extension of basic conditional statements. They allow one if statement to be placed inside another if or else block. This enables programs to make decisions that depend on multiple conditions evaluated in sequence. Nested decision structures are particularly useful when a second condition should only be tested after the first condition has been satisfied.

In real-world programming, many decisions are hierarchical in nature. For example, a system may first check whether a user is registered before checking the user's access level. Nested if statements provide a natural way to represent such layered logic.

3.3.2 General Structure of Nested if Statements

The basic syntax of a nested if statement is shown below:

```

if (condition1) {
    // Executes if condition1 is true
    if (condition2) {
        // Executes if both condition1 and condition2 are true
    }
}

```

Explanation: condition1 is evaluated first. If condition1 is false, the entire nested structure is skipped. If condition1 is true, condition2 is evaluated. The innermost block executes only if all preceding conditions are true.

3.3.3 Applications of Nested if Statements

Nested if statements are commonly applied in:

1. User Authentication Systems – Verifying login credentials before checking permissions.
2. Input Validation – Ensuring prerequisite inputs are valid before processing dependent data.
3. Grading Systems – Categorizing performance based on score ranges.
4. Decision-Based Simulations – Games and interactive applications where choices depend on previous outcomes.

3.3.4 Example: Student Performance Evaluation

```

score = 85
if score >= 70:
    print("Passing grade")
    if score >= 90:
        print("Excellent")
    else:
        print("Good")
else:
    print("Failing grade")

```

Explanation: Scores below 70 result in a failing grade. Scores from 70 to 89 are classified as passing and good. Scores of 90 and above are classified as excellent.

3.3.5 Advantages and Disadvantages

Advantages:

- Enables detailed decision-making.
- Accurately models real-world logic.
- Provides control over complex program flow.

Disadvantages:

- Deep nesting reduces readability.
- Difficult to debug if poorly structured.
- May introduce logical ambiguity.

3.3.6 The Dangling Else Problem

Definition

The dangling else problem occurs when an else clause can logically be associated with more than one if statement. Most programming languages resolve this ambiguity by pairing the else with the closest unmatched if, which may not reflect the programmer's original intention.

Ambiguous Example:

```
if condition1:
    if condition2:
        print("Action A")
else:
    print("Action B")
```

3.4 Logical Operators (&&, ||, !)

3.4.1 Overview

Logical operators allow programmers to combine multiple conditions into a single logical expression. They are essential for constructing meaningful decision-making logic in programs. The three primary logical operators are: AND (&&), OR (||), NOT (!)

3.4.2 The AND Operator (&&)

The AND operator evaluates to true only when all conditions are true.

```
if (age >= 18 && hasID) {
    print("You may enter");
}
```

3.4.3 The OR Operator (||)

The OR operator evaluates to true when at least one condition is true.

```
if (isWeekend || isHoliday) {
    print("No work today!");
}
```

3.4.4 The NOT Operator (!)

The NOT operator reverses the truth value of a condition.

```
if (!isRaining) {
    print("You can go for a walk.");
}
```

3.4.5 Combining Logical Operators

Note: Parentheses are used to control evaluation order and improve clarity.

```
if (age >= 18 && (hasID || isParent)) {
    print("You can enter.");
}
```

```
}
```

3.5 The switch Statement

3.5.1 Introduction

The switch statement is a selection control structure used to execute different blocks of code based on the value of a single variable. It provides a cleaner alternative to long chains of if-else statements when dealing with fixed values.

3.5.2 Syntax of the switch Statement

```
switch (variable) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    default:  
        // statements  
}
```

3.5.3 Example: Day of the Week

```
switch (day) {  
    case 1:  
        print("Monday");  
        break;  
    case 2:  
        print("Tuesday");  
        break;  
    case 3:  
        print("Wednesday");  
        break;  
    case 4:  
        print("Thursday");  
        break;  
    case 5:  
        print("Friday");  
        break;  
    case 6:  
        print("Saturday");  
        break;  
    case 7:
```

```
    print("Sunday");  
    break;  
    default:  
        print("Invalid day");  
}
```

3.5.4 Advantages and Limitations

Advantages:

- Improves readability.
- Simplifies maintenance.
- Efficient for multiple discrete options.

Limitations:

- Fall-through errors if break is omitted.
- Limited data types in some languages.

CHAPTER FOUR

CONTROLLING PROGRAM FLOW WITH LOOPS

(V. I. Osubor, A. R. Usiobaifo and R. O. Osaseri)

1. Introduction to Program Flow and Iteration

In programming, program flow refers to the order in which statements are executed. By default, statements execute sequentially, from top to bottom. However, many real-world problems require repeating actions, such as:

- Displaying numbers from 1 to 100
- Calculating the sum of user inputs
- Processing records in a database
- Generating multiplication tables

To handle repetition efficiently, programming languages provide loops. A loop is a control structure that executes a block of code repeatedly as long as a specified condition is true.

2. Types of Loops

Most programming languages provide three primary loop structures:

1. WHILE loop – condition tested before execution
2. FOR loop – counter-controlled repetition
3. DO-WHILE loop – condition tested after execution

Each loop serves a specific purpose and is suitable for different types of problems. In Java, loops allow repeated execution of a block of code while a condition remains true. Java provides three loop constructs: while, for and do-while. Java also provides loop control statements: break and continue.

4.1 The WHILE Loop

1 Definition

A WHILE loop repeatedly executes a block of code as long as a condition remains true. The condition is evaluated before each iteration.

2 Syntax

```
while (condition) {  
    statements;  
}
```

Flow Description:

1. The condition is evaluated.
2. If the condition is true, the loop body executes.

3. After execution, the condition is checked again.
4. If the condition becomes false, the loop terminates.

Example 1: Printing Numbers from 1 to 5

```
public class WhileExample1 {  
    public static void main(String[] args) {  
        int counter = 1;  
        while (counter <= 5) {  
            System.out.println(counter);  
            counter++;  
        }  
    }  
}
```

Example 2: Sum of First N Natural Numbers

Explanation: Initialization: counter = 1, Condition: counter <= 5, Update: counter increases by 1 each iteration

```
import java.util.Scanner;  
public class WhileSum {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter a number: ");  
        int n = scanner.nextInt();  
        int sum = 0;  
        int i = 1;  
        while (i <= n) {  
            sum += i;  
            i++;  
        }  
    }  
}
```

Common Pitfall: Infinite while Loop

Incorrect Code (Infinite Loop):

```
// **Incorrect Code**  
int i = 1;  
while (i <= 5) {  
    System.out.println("Hello");  
    // i is never incremented – infinite loop  
}
```

Corrected Code:

```
// **Corrected Code**
int i = 1;
while (i <= 5) {
    System.out.println("Hello");
    i++;
}
```

4.2 The FOR Loop

Definition

A FOR loop is used when the number of iterations is known in advance. It combines initialization, condition, and update in one line.

Syntax

```
for (initialization; condition; update) {
    statements;
}
```

Iterative Problem 3: Printing Even Numbers from 2 to 10

```
for (int i = 2; i <= 10; i = i + 2) {
    print(i)
}
```

Explanation: Initialization: $i = 2$, Condition: $i \leq 10$, Update: $i = i + 2$

Example 4: Factorial of a Number

```
import java.util.Scanner;

public class FactorialForLoop {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = scanner.nextInt();
        long factorial = 1;
        for (int i = 1; i <= n; i++) {
            factorial *= i;
        }
        System.out.println("Factorial = " + factorial);
        scanner.close();
    }
}
```

Common Pitfall: Wrong Loop Condition

Incorrect Code:

```
// **Incorrect Code**  
for (int i = 1; i >= 5; i++) {  
    System.out.println(i);  
}
```

Problem: Condition is false at the start. Loop body never executes.

Corrected Code:

```
// **Corrected Code**  
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

4.3 The DO-WHILE Loop

A DO-WHILE loop executes its body at least once, regardless of the condition. The condition is evaluated after execution.

Syntax

```
do {  
    statements;  
} while (condition);
```

Infinite 'for' Loop Example:

```
// This loop never terminates because 'i' decreases.  
for (int i = 1; i <= 5; i--) {  
    System.out.println(i);  
}
```

Example 5: Menu-Driven Program

```
import java.util.Scanner;  
public class DOWhileMenu {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        int choice;  
        do {  
            System.out.println("1. Add");  
            System.out.println("2. Subtract");  
            System.out.println("3. Exit");  
            System.out.print("Choose an option: ");  
            choice = scanner.nextInt();  
        } while (choice != 3);  
        System.out.println("Program exited.");  
        scanner.close();  
    }  
}
```

```
}  
}
```

Explanation:

- The menu appears at least once.
- Loop continues until option 3 is selected.

4.4 Loop Control Statements: break and continue

4.4.1 The break Statement

break terminates the loop immediately, regardless of the condition.

Example: Stop Loop When Number Equals 5

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                break;  
            }  
            System.out.println(i);  
        }  
    }  
}
```

4.4.2 The continue Statement

continue skips the current iteration and moves to the next one.

Example: Skip Even Numbers

```
public class ContinueExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i % 2 == 0) {  
                continue;  
            }  
            System.out.println(i);  
        }  
    }  
}
```

Pitfall with 'continue' in 'while' loop — Incorrect Code (Infinite Loop):

```
int i = 1;  
while (i <= 5) {  
    if (i == 3) {
```

```

        continue; // i is never incremented when i==3 – infinite loop
    }
    System.out.println(i);
    i++;
}

```

Corrected Code:

```

int i = 1;
while (i <= 5) {
    if (i == 3) {
        i++;
        continue;
    }
    System.out.println(i);
    i++;
}

```

4.5 Nested Loops

A nested loop is a loop inside another loop. The inner loop executes completely for each iteration of the outer loop.

Example 6: Multiplication Table (3 x 3)

```

public class MultiplicationTable {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                System.out.print(i * j + " ");
            }
            System.out.println();
        }
    }
}

```

Example 7: Star Pattern

```

public class StarPattern {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}

```

```

    }
}

```

Common Pitfall: Reusing Loop Variables

Incorrect Code:

```

for (int i = 1; i <= 3; i++) {
    for (int i = 1; i <= 3; i++) { // Compilation error
        System.out.println(i);
    }
}

```

Correct Code:

```

for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(i + " * " + j + " = " + (i*j));
    }
}

```

Summary

Structure	Use When	Best Loop
while	Unknown iterations	while
for	Known iterations	for
do-while	Execute at least once	do-while
Nested for	Multi-dimensional problems	Nested for
break	Exit loop early	Any
continue	Skip current iteration	Any

Lab Exercises (Chapter 4):

1. Java SE 1.7 supports 'while', 'for' and 'do-while'.
2. Variables must be properly typed and initialized.
3. Scanner is a class commonly used for reading user input.
4. Never forget to update statements to avoid infinite loops.
5. Print numbers from 10 to 1 using a 'while' loop.
6. Print a 5 x 5 multiplication table using a 'for' loop.
7. Write a combination of 1 to 3 between 1 and 100 using a 'for' loop.
8. Write a combination of 1 to 3 between 1 and 100 using a 'do-while' loop.
9. Use a do-while loop to simulate code.

CHAPTER FIVE

INTRODUCTION TO OBJECTS AND CLASSES

(S. Konyeha, M. S. U. Osagie and B. Odunyo)

5.1 Objects as Data Abstractions

In the world of Java, object-oriented programming (OOP) is the core paradigm used to organize software. At its foundation, Java treats everything as an object. An object is defined as any entity that possesses both a state and behavior. These entities can be physical, such as a chair or a pen, or logical, such as a bank account or a data connection.

Data abstraction is the process of hiding internal implementation details while showing only the essential functionality to the user. When we use objects as data abstractions, we focus on what an object does rather than how it works internally. For instance, when you use a mobile phone to make a call, you understand the functionality (the 'what') without needing to know the complex internal processing (the 'how').

In Java, objects provide a way to bind code and data together into a single unit, a concept known as encapsulation. This 'wrapping' ensures that the data is protected and can only be accessed through the object's defined behaviors, much like a medicinal capsule protects the medicine inside.

5.2 Defining a Class: States (Fields) and Behaviors (Methods)

If an object is a real-world entity, then a class is the logical entity that defines the common properties and actions for a group of similar objects. To define a class in Java, we use the class keyword.

States (Fields): The state of an object is represented by its fields, also known as instance variables. A variable is a container that holds a value while the program is executed. It is a container that is in the class definition, these are typically private, allowing different objects of the class to have different states.

Behaviors (Methods): The behavior of an object is defined by its methods. A method is a block of code used to perform specific actions or interact with an object's state.

Constructors: Special code blocks called during object creation to allocate memory.

Definition of Terms

Objects: Defined as any entity (physical or logical) possessing state and behavior.

Defining Classes: A class is a 'logical entity' and a template for a collection of objects.

Encapsulation: The binding or 'wrapping' of code and data together into a single unit, a concept that shows functionality to the user.

Data Abstraction: The process of hiding internal details while showing essential functionality to the user.

States (Fields): Represented by instance variables declared inside a class but outside the method body.

Behaviors (Methods): Blocks of code used to perform specific actions or interact with an object's state.

Constructors: Special code blocks called during object creation to allocate memory.

Example: A Blueprint for a Simple Student Object

```
// A blueprint for a simple Student object
class Student {
    // States: Fields (Instance Variables)
    String name;
    int id;

    // A constructor to initialize the object
    Student(int i, String n) {
        id = i;
        name = n;
    }

    // Behavior: A method to display data
    void display() {
        System.out.println(id + ". " + name);
    }
}
```

The Role of Constructors:

When we create an instance of a class, a constructor is invoked. The constructor initializes the object and allocates memory for it in the system. In Java, methods often have a return type such as void (if no value is returned) or a specific data type. The constructor is a special block of code that often have a return type void (if no value is returned) or a specific data type. The constructor, however, does not have a return type, not even void.

The this keyword can also be used to call one constructor from another constructor within the same class, which helps avoid code duplication.

5.5 Encapsulation: The Principle of Data Hiding

Encapsulation is one of the core principles of Object-Oriented Programming (OOP). It refers to the practice of bundling data (fields) and the methods that operate on that data into a single unit called a class, while restricting direct access to some of the object's internal details. In simpler terms, encapsulation means hiding how an object works internally and exposing only what is necessary for the user to interact with it safely.

A useful real-world analogy is a bank ATM. When you withdraw money, you interact with a simple interface: insert your card, enter your PIN, and select an amount. You are not allowed to directly access the bank's database or change your account balance yourself. The internal data and operations are hidden, and only approved actions are allowed. This is exactly how encapsulation works in programming.

In Java, encapsulation is achieved primarily using access modifiers, especially private, public, and sometimes protected. Fields (instance variables) are usually declared as private, meaning they cannot be accessed directly from outside the class. Instead, controlled access is provided through public methods, commonly known as getter and setter methods.

Example: Student Class Without Encapsulation

```
class Student {  
    public String name;  
    public int age;  
}
```

In this design, any part of the program can directly modify the name and age fields:

```
Student s = new Student();  
s.age = -10; // Invalid but allowed
```

This is dangerous because it allows invalid or inconsistent data. Encapsulation solves this problem by hiding the fields and controlling access:

```
class Student {  
    private String name;  
    private int age;  
  
    public void setAge(int age) {  
        if (age > 0) {  
            this.age = age;  
        }  
    }  
}
```

Now, the age field cannot be modified directly. The setAge() method ensures that only valid values are accepted. This makes the program more reliable, secure, and easier to maintain.

Encapsulation offers several important benefits:

1. It protects data integrity by preventing accidental or unauthorized modifications.
2. It improves maintainability, because internal changes to a class do not affect other parts of the program as long as the public interface remains the same.
3. It supports modularity, allowing developers to work on different classes independently.

In summary, encapsulation enforces the idea that objects should manage their own data, and external code should interact with objects only through well-defined methods. Just as you trust a device or system to work correctly without knowing its internal details, encapsulation allows programmers to build robust systems by hiding complexity and exposing simplicity.

5.6 Constructors (Default and Parameterized) and the this Keyword

A constructor is a special method in a class that is automatically called when an object is created. Its main purpose is to initialize the object's fields with appropriate starting values. In Java, a constructor has the same name as the class and does not have a return type, not even void.

There are two commonly used types of constructors: default constructors and parameterized constructors.

Default Constructor

A default constructor is a constructor that takes no parameters. If a class does not explicitly define any constructor, Java automatically provides a default constructor that initializes fields with default values (such as 0 for integers and null for objects). However, programmers often define their own default constructors to assign meaningful initial values.

Example of a default constructor:

```
class Car {  
    String brand;  
    int year;  
  
    Car() {  
        brand = "Unknown";  
        year = 0;  
    }  
}
```

When an object is created using this constructor:

```
Car c1 = new Car();
```

The fields `brand` and `year` are automatically initialized with predefined values. This ensures that the object starts in a valid and predictable state.

Parameterized Constructor

A parameterized constructor, on the other hand, accepts one or more parameters. It allows the programmer to supply specific values at the time an object is created.

Example of a parameterized constructor:

```
class Car {  
    String brand;  
    int year;  
  
    Car(String b, int y) {  
        brand = b;  
        year = y;  
    }  
}
```

Now, objects can be created with different initial values:

```
Car c2 = new Car("Toyota", 2022);  
Car c3 = new Car("Honda", 2020);
```

This approach makes object creation more flexible and meaningful, as each object can represent a unique real-world entity.

The this Keyword

At this point, it is important to introduce the `this` keyword. The `this` keyword is a reference to the current object, the object that is currently being created or used. It is commonly used to distinguish between instance variables and parameters that have the same name.

Consider the following constructor:

```
class Student {  
    String name;  
    int age;
```

```

Student(String name, int age) {
    name = name; // This is actually wrong!
    age = age;
}
}

```

This code looks correct but is actually incorrect. The assignments do nothing meaningful because the parameter names hide the instance variables. The `this` keyword fixes this problem:

```

class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

```

Here, `this.name` refers to the instance variable, while `name` refers to the parameter. This makes the assignment clear and correct.

Lab Exercise: Implementing a Bank Account Class

Campus Lab: Simulating a Bank Account and Examining OOP Concepts

Objective: Implement a basic `BankAccount` class. This class should simulate a bank account with the ability to work a balance and perform withdrawals and deposits. Create a class named `BankAccount`. It must include two fields: `accNo` and `balance`.

Task Description:

Design and implement a `BankAccount` class. The class should provide the following behaviors: `deposit(double amount)` — Adds the specified amount to the current balance. `withdraw(double amount)` — Subtracts the specified amount from the balance. For this lab, assume the balance can go negative (no validation required). `display()` — Prints the account number and the current balance to the console.

Step 1: Define the Class and Fields

```

class BankAccount {
    int accNo;
    double balance;
}

```

Step 2: Implement Methods (Behaviors)

```

// Methods (Behaviors)
void deposit(double amount) {
    balance = balance + amount;
    System.out.println("Deposited: " + amount);
}

void withdraw(double amount) {

```

```

        balance = balance - amount;
        System.out.println("Withdrawn: " + amount);
    }

    void display() {
        System.out.println("Account No: " + accNo + " | Balance: " + balance);
    }
}

```

Step 3: Testing the Class

```

public class TestBank {
    public static void main(String[] args) {
        // Creating the first object
        BankAccount a1 = new BankAccount();
        a1.accNo = 10101;
        a1.balance = 1000.0;

        // Creating the second object
        BankAccount a2 = new BankAccount();
        a2.accNo = 10102;
        a2.balance = 500.0;

        // Testing behaviors for account 1
        a1.display();
        a1.deposit(500.0);
        a1.withdraw(200.0);
        a1.display();

        System.out.println("-----");

        // Testing behaviors for account 2
        a2.display();
        a2.deposit(100.0);
        a2.display();
    }
}

```

Review Questions (Chapter 5):

1. What is the difference between private and public access modifiers in the context of encapsulation?
2. What is the purpose of a constructor in Java?
3. Explain how getter and setter methods help enforce data validation. Provide an example.
4. What is encapsulation and how does it improve software maintainability and modularity?
5. Differentiate between default and parameterized constructors.
6. Why does a constructor not have a return type?
7. Explain the role of the this keyword.

Lab Exercises (Chapter 5):

1. Create a Student class with private fields name (String) and age (int). Add public getter and setter methods for each field. Modify the setter method for age so that it only allows positive values; reject negative numbers.
2. Create a class with both default and parameterized constructors. Use the this keyword to resolve naming conflicts.
3. Create a class with both default and parameterized constructors.
4. Create an Employee class with private fields id, name, and salary. Write getter and setter methods, and test them by creating multiple objects.
5. Use the this keyword to resolve naming conflicts.
6. Demonstrate constructor overloading with three constructors.

CHAPTER SIX

STRING PROCESSING AND SIMPLE I/O

(E. Nweli and B. Odunyo)

6.0 Introduction

Modern interaction with computer systems is fundamentally centered around text. Whether entering a name during authentication, searching for information online, exchanging messages, reading emails, or issuing commands to applications, the underlying elements in these activities is the use of strings. Strings are sequences of characters that represent meaningful textual data, and in programming particularly in Java, they serve as a primary medium through which programs communicate with users and external systems. From simple calculations to registration platforms and mobile applications, the ability to read, manipulate, and display textual information is essential.

This chapter introduces one of the core concepts in programming: string processing and simple input/output operations. Mastery of these skills enables the development of programs that interact effectively with users, rather than merely executing code. These capabilities elevate software from basic code execution to practical, user-friendly applications.

The chapter begins by examining the nature of strings, how Java stores and manages them, and the reasons strings operate differently from numerical data types. It then covers key string operations, including concatenation, character extraction, searching, comparison, and formatting. The latter part of the chapter introduces the Scanner class to collect data from the keyboard and display information on the console. This establishes the foundation for building interactive, text-driven programs written in clear and readable Java code.

By the end of the chapter, readers will be equipped to construct programs that accept and process textual input, perform logical operations based on that input, and present results in a clear and professional format. The knowledge gained here forms a critical foundation for more advanced topics in Java and computer programming generally, as text processing remains a central component of nearly all software systems.

6.1 Strings as Character Sequences

A String in Java is a sequence of characters indexed from zero. This makes it possible to access individual characters and extract specific portions of text. For example in the string 'Java':

Chars:	J	a	v	a
Index:	0	1	2	3

This structure allows programs to access individual characters or extract specific portions of text using methods provided by the String class.

After Concatenation:

```
String firstName = "Ada";  
String lastName = "Obi";
```

```
// Using the + operator:  
String fullName = firstName + " " + lastName;  
// Updated Name: Ada Obi
```

While both approaches produce valid strings, the literal form is generally preferred because it takes advantage of the String Pool and is more efficient. The new `String()` approach forces Java to create a new String object outside the pool, even if the same value exists, which is generally less efficient and rarely necessary in beginner-level programming.

6.1.2 String Immutability

A key characteristic of strings in Java is that they are immutable. Immutability means that once a string object is created, its value cannot be changed. Any operation that appears to modify a string such as concatenation, replacement, or conversion to uppercase actually creates a new string object rather than altering the original one.

This simple example shows the flexibility of strings: they can hold text, be combined using the `+` operator, and be displayed directly to the user.

Example:

```
String name = "Ada";  
String updatedName = name + " Obi";  
// After execution:  
// The original string "Ada" remains unchanged.  
// A new string "Ada Obi" is created and stored in updatedName.
```

Immutability offers several important advantages:

- A new string 'Ada Obi' is created and stored in updated Name.
- The original string 'Ada' remains unchanged.
- It ensures strings are safe to share across different parts of a program.
- It supports memory optimization through the String Pool.
- It prevents unintended modifications to text values, reducing bugs.

6.2 Essential String Methods

As strings form the foundation of many programming tasks such as reading input, generating output, storing names, formatting messages, and managing user interactions, understanding how they work is essential for progressing in Java programming. The subsequent sections build on this foundation by exploring key operations that allow developers to process text efficiently and correctly.

After creating strings, Java provides a rich set of built-in methods through the `String` class to examine and manipulate textual data. These methods make it possible to measure text length, access individual characters, extract portions of a string, search for specific content, compare text values, and transform text into different formats. Because text processing appears in nearly all software systems, familiarity with these essential methods is critical for effective Java programming. This section introduces five of the most commonly used and important string methods: `length()`, `charAt()`, `substring()`, `indexOf()`, and `equals()`.

6.2.1 The `length()` Method

The `length()` method returns the number of characters contained in a string, including spaces and symbols.

Outputs:

```
18
String course = "Computer Science";
```

6.2.2 The `charAt()` Method

The `charAt()` method retrieves a single character from a string based on its index position. Indexing in Java begins at zero.

Conceptual Illustration:

```
J a v a
0 1 2 3
```

Output: 'Java'

The `charAt()` method is frequently used for validation tasks, such as checking whether a password meets a minimum length requirement or determining how many characters a user has entered.

Index Representation:

```
Index: 0 1 2 3
String: J a v a
char letter = language.charAt(2);
// Output: letter = "v"
```

6.2.3 The `substring()` Method

The `substring()` method extracts a portion of a string. It works by specifying a starting index and, optionally, an ending index.

This method is useful when analyzing individual characters one at a time in a loop, including checking how many times a specific character appears in a string.

In this example:

- The starting index (0) is inclusive.
- The ending index (4) is exclusive.

Visual Explanation:

```
"Computer"
C o m p u t e r
0 1 2 3 4 5 6 7
```

Output: Comp

```
String text = "Computer";
String part = text.substring(0, 4);
// Output: 'Comp'
```

6.2.4 The `indexOf()` Method

The `indexOf()` method searches for a character or sequence of characters within a string and returns the index of its first occurrence. If the target is not found, the method returns -1.

The `indexOf()` method is commonly used to extract usernames from email addresses, file extensions from filenames, or meaningful parts of structured text.

Illustration:

```
String sentence = "Learning Java is fun";  
int position = sentence.indexOf("Java");  
// Output: position = 9
```

6.2.5 The `equals()` Method

This method is useful for locating specific words, validating input upon user input, or determining whether a string contains a certain sequence of characters.

It is important to note that the `==` operator should not be used for string comparison. It compares memory references rather than actual text values.

```
String s1 = "Java";  
String s2 = "Java";  
// True
```

Comparison Summary:

Method	Purpose
<code>==</code>	Compares memory locations
<code>equals()</code>	Compares string content

4.1.5 Summary of Essential String Methods

Method	Description	Example
<code>length()</code>	Returns number of characters	"Java".length() → 4
<code>charAt(i)</code>	Returns character at index	"Java".charAt(1) → a
<code>substring()</code>	Extracts part of string	"Java".substring(1,3) → av
<code>indexOf()</code>	Finds position of first occurrence	"Java".indexOf("v") → 2
<code>equals()</code>	Compares string contents	"Java".equals("Y") → true

Understanding and correctly applying these essential string methods enables programmers to inspect, manipulate, and analyze textual data effectively.

6.3 Introduction to the Scanner Class for User Input

Before using the Scanner class, it must be created and imported into the program. Once imported, a Scanner object is created and linked to `System.in`, which represents standard keyboard input.

6.3.1 Importing and Creating a Scanner Object

Before using the Scanner class, it must be imported into the program. Once imported, a Scanner object is created and linked to `System.in`, which represents standard keyboard input.

```
import java.util.Scanner;  
  
// This object, commonly named input or scanner, acts as a channel through which  
// the program reads data entered by the user.
```

```
Scanner input = new Scanner(System.in);
```

Conceptual Diagram:

```
Keyboard --> Scanner Object --> Java Program
```

6.3.2 Reading String Input

The Scanner class provides several methods for reading text. The most commonly used method for string input is `nextLine()`, which reads an entire line of input, including spaces.

```
Scanner input = new Scanner(System.in);  
System.out.print("Enter your full name: ");  
String name = input.nextLine();  
System.out.println("Welcome, " + name + "!");
```

Sample Output:

```
Enter your full name: Adaeze Obi  
Welcome, Adaeze Obi!
```

6.3.3 Reading Text Input with `nextLine()`

The `nextLine()` method reads an entire line of input, including spaces, until the user presses the Enter key. It is the most appropriate method for reading sentences, names, or any text that may contain spaces.

```
input.nextLine();  
System.out.print("Enter your department name: ");  
String department = input.nextLine();  
System.out.println("Department: " + department);
```

Sample Output:

```
Enter your department name: Computer Science  
Department: Computer Science
```

6.3.3 Reading Numeric Input

The `nextInt()` method focuses on three of the most commonly used Scanner methods: `nextInt()`, `nextDouble()`, and `nextLine()`. Understanding how these methods work with one another is essential for building reliable and interactive Java programs.

6.3.1 Reading Integer Values with `nextInt()`

The `nextInt()` method is used to read whole numbers (integers) entered by the user. When this method is called, the program waits until the user enters a valid integer and presses the Enter key.

```
Scanner input = new Scanner(System.in);  
System.out.print("Enter the number of students: ");  
int students = input.nextInt();
```

Sample Output:

```
Enter the number of students: 40  
Number of students: 40
```

6.4 Understanding the Input Buffer

When reading input, Java temporarily stores what the user types in an input buffer. Methods such as `nextInt()` and `nextDouble()` read only the numeric value and leave the newline character (`\n`) in the buffer. This can cause unexpected behavior when `nextLine()` is used immediately afterward.

Illustration of the Input Buffer:

```
User Input: 25
```

```
Stored in Buffer: 25\n
```

If `nextLine()` is called immediately, it may read the leftover newline instead of waiting for new input.

6.4.5 Mixing Numeric and String Input Correctly

Incorrect Approach:

```
System.out.print("Enter your age: ");
int age = input.nextInt();
System.out.print("Enter your name: ");
String name = input.nextLine(); // Skipped!
```

Correct Approach:

```
System.out.print("Enter your age: ");
int age = input.nextInt();
input.nextLine(); // Clear the buffer
System.out.print("Enter your name: ");
String name = input.nextLine();
```

This extra call to `nextLine()` consumes the leftover newline character and ensures proper input reading.

6.4.2 Reading Decimal Values with `nextDouble()`

The `nextDouble()` method is commonly used for values such as age, quantity, count, or selection options in menu-driven programs. For values that include decimal points, such as averages, prices, or measurements, the `nextDouble()` method is used.

```
System.out.print("Enter the course average score: ");
double average = input.nextDouble();
```

Sample Output:

```
Enter the course average score: 78.5
```

```
Average Score: 78.5
```

This method ensures that numeric input is stored accurately in a variable of type `double`.

6.4.6 Complete Example: Reading Multiple Data Types

```
import java.util.Scanner;
import java.util.Scanner;

public class StudentData {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
```

```
System.out.print("Enter student name: ");
String name = input.nextLine();

System.out.print("Enter student age: ");
int age = input.nextInt();

System.out.print("Enter student GPA: ");
double gpa = input.nextDouble();

}
}
```

Sample Output:

```
Enter student name: Amina
Enter student age: 19
Enter student GPA: 3.75
```

6.3.6 Common Beginner Mistakes When Using Scanner

Mistake	Evaluation	Correct Practice
Forgetting to import Scanner	Causes compilation error	import java.util.Scanner;
Mixing nextInt() and nextLine()	Leads to skipped input	Clear buffer with nextLine()
Not prompting the user	Causes confusion	Always display prompts
Closing Scanner too early	Prevents further input	Close only at program end

6.3.7 Best Practices for Using Scanner

- Close the Scanner at the end of the program to release resources: input.close();
- Use meaningful variable names for Scanner objects (e.g., input, scanner).
- Prompt users clearly before reading input.
- Validate user input where necessary.
- Use nextLine() when reading full lines of text.

6.4 Reading Different Data Types from the Console

The Scanner class provides a simple and effective way to introduce user interaction in Java programs. By combining Scanner-based input with string processing techniques, programs can collect, analyze, and respond to user data in a structured and meaningful manner. This forms a critical foundation for building interactive console applications and more advanced systems in Java programming.

User input in real-world applications is rarely limited to a single type of data. Programs often need to read numbers, text, and combinations of both in order to perform meaningful operations. In Java, the Scanner class provides specialized methods for reading different data types, allowing programs to interpret user input correctly based on the expected form of data.

6.4.7 Summary of Scanner Input Methods

Method	Data Type Read	Typical Use Case
--------	----------------	------------------

nextInt()	Integer	Age, count, menu option
nextDouble()	Decimal number	Scores, prices, averages
nextLine()	Full line of text	Names, sentences, addresses

6.4.8 Common Errors and Best Practices

Common Errors:

- Attempting to read text with nextInt() or nextDouble()
- Forgetting to clear the input buffer
- Not validating user input
- Assuming the user will always enter valid data

Best Practices:

- Always prompt the user clearly
- Use the correct Scanner method for the expected data type
- Clear the input buffer when switching between numeric and text input
- Close the Scanner at the end of the program

Reading different data types accurately is a fundamental requirement for interactive Java programs. By correctly applying nextInt(), nextDouble(), and nextLine(), and handling features such as input validation, menu-driven systems, and data-driven applications.

6.5 A Complete Example Program

```
import java.util.Scanner;

public class UserInputDemo {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = input.nextLine();

        System.out.print("Enter your age: ");
        int age = input.nextInt();
        input.nextLine(); // Clear the buffer

        System.out.print("Enter your score: ");
        int score = input.nextInt();

        System.out.println("Student Name: " + name);
        System.out.println("Score: " + score);
    }
}
```

This ensures that the program reads input correctly.

6.6 Lab Exercises

Have a full name as input.

Check if it is a palindrome.

Read two strings and check whether they are equal using the equals() method.

Read a sentence and remove all leading and trailing spaces.

Prompt the user for an email address and display the index position of the '@' symbol.

Ask the user for a sentence and check whether it contains the word 'Java'.

Read a sentence and check whether it contains 'Java'.

References

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.

Deitel, P., & Deitel, H. (2023). Java: How to Program (12th ed.). Pearson Education.

Horstmann, C. S. (2022). Core Java Volume I — Fundamentals (12th ed.). Pearson Education.

Oracle. (2024). The Java™ Tutorials: Encapsulation and Access Control. Oracle Corporation.
<https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>

Oracle. (2021). Java SE 17 documentation. <https://docs.oracle.com/en/java/javase/17/>

Schildt, H. (2021). Java: The complete reference (12th ed.). McGraw-Hill Education.